

Autonóm navigációs stack Unitree Go2 robothoz

1 Konceptió

A `fast_lio_go2` repo egy konténerizált, OptiTrack nélküli autonóm navigációs keretrendszer a Unitree Go2 robothoz. A robot beépített Unilidar L1/L2 LiDAR szenzorait használva valós idejű állapotbecslést, térképezést és megadott pózba való navigációt (point-pose) valósít meg moduláris ROS 2 architektúrán keresztül.

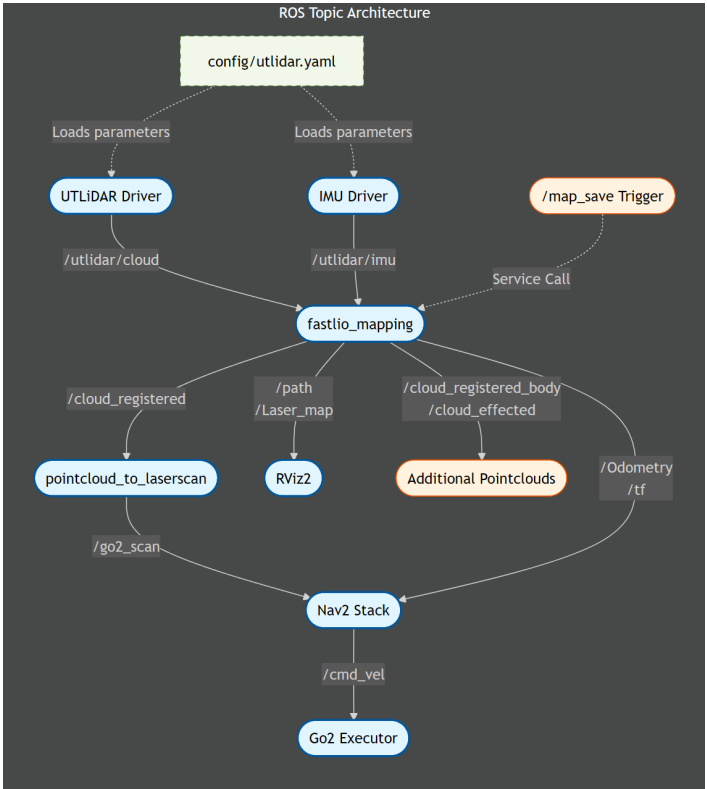
2 Modulok

Az alábbi alapsomagok és komponensek vannak integrálva:

- **FAST-LIO (módosított):** Állapotbecslést és LiDAR alapú, valós idejű térképezést végez. A Unilidar-ból érkező adatfolyammal való kompatibilitás megteremtése a `point_lio_unilidar` repo-ból lett átvéve. A módosítások a forrásfájlokban találhatóak, ezen kiegészítésekkel támogatva vannak az egyedi pont-időbélyegek és adatszerkezetek.
- **Nav2 szoftvercsomag:** Felelős a globális útvonaltervezésért, a lokális trajektóriakövetésért, valamint a costmap-ek vetítéséért a térképből kinyert 2D lézerszkennelések alapján. A `go2_nav2_params.yaml` fájlban keresztül állítható.
- **Unitree SDK2 (execution.py):** Egy önálló Python végrehajtó node, amely a sebességparancsokat figyeli, és a Unitree fejlesztői csomag segítségével a Go2 sport kliens felé továbbítja azokat.
- **Főbb kiegészítések:**
 - Egyedi indítási szekvencia, ami betölti az összes modult egy parancsos indítással. (`master_launch.py`)
 - Illesztett koordináta-transzformációk a megfelelő megjelenítéshez. (`mapping.launch.py`)
 - Személyre szabott RViz konfiguráció a szükséges adatfolyamok felügyeletéhez és parancskiadáshoz. (`fastlio.rviz`)
 - A navigációhoz szükséges egy előre felvett globális costmap, a *Workflow 1* tartalmazza a szükséges lépéseket.

Adatfolyam áttekintő

Publisher	Topic	Adatfolyam és Consumer
UTLiDAR / IMU illesztők	<code>/utlidar/cloud</code> , <code>/utlidar/imu</code>	Nyers szenzoradatok közvetlen továbbítása a FAST-LIO felé.
FAST-LIO	<code>/cloud_registered</code>	Torzítás nélküli pontfelhő a lézerszkenneléshez.
Pointcloud to laserscan	<code>/go2_scan</code>	Síkbeli szkenn a Nav2 costmap-jei számára
FAST-LIO	<code>/Odometry</code> , <code>/tf</code>	Térbeli állapotbecslés és transzformációk a Nav2 és RViz felé.
Nav2 Szabályozó	<code>/cmd_vel</code>	A végrehajtó node által feldolgozott sebességcélok.
Végrehajtó node	N/A (Unitree SDK2 API)	Közvetlen SDK hívások a robot mozgásának irányítására.



Moduláris felépítés: Mivel a modulok szabványos ROS 2 témákon, szolgáltatásokon és TF fákön keresztül kommunikálnak, az egyes blokkok egymástól függetlenül cserélhetők.

3 Navigációs szabályozási hurok

Amikor a rendszer egy célpózt kap (részletek: *Workflow 2*), az architektúra az alábbi folyamaton futtatja végig a parancsot:

1. **Globális útvonalgenerálás:** A viselkedésfa-navigátor elindítja a globális tervezőt, amely egy ütközésmentes útvonalat számítki a globális costmap-en.
2. **Lokális trajektóriaszabályozás:** A szabályozó szerver mintavételezi az útvonalat, és lokális sebességparancsokat generál a szabályozott "pure pursuit" algoritmus segítségével, figyelembe véve a lokális costmap-en megjelenő dinamikus akadályokat.
3. **Végrehajtás:** Az eredményül kapott `/cmd_vel` üzeneteket az `execution.py` feldolgozza, sebességi holtávokon szűri át, majd végrehajtja azokat a Go2 sport client segítségével.
4. **Háttérben futó biztonsági hurkok:** Ezzel párhuzamosan a FAST-LIO magas frekvencián frissíti az állapotbecsléseket, a costmap-ek törlik a dinamikus akadályokat, és a végrehajtó egy 0,5 s-os watchdog időzítővel biztonságosan megállítja a robotot, ha a tervezőtől érkező parancsok megszakadnak.

4 Dockerizációs protokoll

A környezet egy Docker konténerben fut, amelyet a Docker Compose kezel.

- **Alapkomponensek:** Tartalmaz minden fejlesztői csomagot ami a modulok futtatásához szükséges.
- **Hálózati konfiguráció:** A csomagot futtató számítógép hálózati interfésze a `NETWORK_INTERFACE` környezeti változón keresztül adódik át, a konténer építéskor és futtatás előtt is állítható.
- **DDS réteg:** Aktiválja a CycloneDDS-t egy XML konfigurációs URI-n keresztül, ami a kommunikációt biztosítja a kutya és a számítógép között.

5 Főbb hangolható paraméterek és Nav2 viselkedés

- **FAST-LIO paraméterek** (`utlidar.yaml`):
 - alulmintavételezés sűrűsége
 - voxel felbontás
 - vakzóna mérete
 - kovarianciák (Kálmán-szűrőhöz)
 - torzítás (Kálmán-szűrőhöz)
- **RViz konfiguráció** (`fastlio.rviz`): Egyéni vizualizáció, ami megjeleníti az egyedi TF fákat, a globális costmap-et, a lézerszkenneléseket és a statikus lézertérképeket.
- **Nav2 paraméterek** (`go2_nav2_params.yaml`):
 - robot kiterjedése
 - előretekintési távolság
 - lineáris sebesség
 - szögsebesség
 - costmap inflációs sugara (lokális és globális)
- **Nav2 Behaviors és Modellek:**
 - **AMCL (Adaptív Monte Carlo Lokalizáció):** A rendszer az AMCL modult használja a 2D globális ráctérképen történő való idejű helyzetmeghatározáshoz.
 - **Robot modell:** A rendszer a `nav2_amcl::OmniMotionModel` kinematikai modellt használja, amely támogatja az oldalirányú (holonomikus) mozgásokat.
 - **Trajektória algoritmus:** A `RegulatedPurePursuitController` felel a lokális útvonalkövetésért, amely az előretekintési távolság alapján szabályozza a lineáris és szögsebességeket.
 - **Spin (Forgás):** A robot helyben forogva frissíti a környezet 360°-os képét, segíti a hamis akadályok törlését a costmap-ből.
 - **BackUp (Tolatás):** Rövid hátrafelé mozgás, ha a robot túlságosan megközelít egy akadályt.
 - **DriveOnHeading (Írányított haladás):** Beavatkozás, amely során a robot egy adott szög mentén vakon halad, általában szűk terekből való kiszabadulásakor.
 - **Wait (Várakozás):** A robot megáll és várakozik, hogy a dinamikus akadályok elhagyják az útvonalat.